path-finder command flow

The happy path of the installed command, from `main` to the script or the window, with one line per function. Covers `path_finder/cli.py`, `repo.py`, and `window.py`. Written 2026-10-21.

What this layer is for

The `path-finder` command holds no application logic. It answers three questions and then gets out of the way: where is the code, which script and arguments to run, and for `window`, how to open the page. Everything else happens inside `discovery/discover.py` and `viz/watcher.py`, which it runs as files.

The call map

```
path-finder <command> [args]
│
└─ main(argv)                       cli.py   Reads argv. No args or --help: print usage(), exit 0.
   │                                          Otherwise calls dispatch; RepoNotFound becomes exit 1.
   │
   └─ dispatch(command, arguments) cli.py   Finds the code folder once, then branches on the command.
      │
      ├─ find_code_folder()        cli.py   Checkout first, bundled copies second.
      │  └─ find_repo()            repo.py  First candidate that passes is_repo, else RepoNotFound.
      │     ├─ candidate_folders() repo.py  Yields in order: $PATH_FINDER_REPO, the package's parent,
      │     │                                the current folder, then each parent of it.
      │     └─ is_repo(folder)     repo.py  True when discovery/discover.py and viz/watcher.py exist.
      │  (on RepoNotFound)
      │  └─ vendored_copies_present()       is_repo on path_finder/_vendor, then use VENDOR_DIR.
      │
      ├─ A. discovery command: identify  self  list  done
      │  ├─ discover_script(repo)  repo.py  repo/discovery/discover.py
      │  ├─ with_default_root()    cli.py   For list and done without --root, append --root <cwd>.
      │  │  └─ has_root_flag()     cli.py   Did the user already type --root or --root=?
      │  └─ run_script_in_place()  cli.py   Sets sys.argv to [script, command, *args] and runs the
      │                                      file as __main__. The script's own argparse takes over.
      │
      ├─ B. watcher command: serve  state  replay  map  tail
      │  ├─ watcher_script(repo)   repo.py  repo/viz/watcher.py
      │  ├─ with_default_root()    cli.py   For serve and map without --root, append --root <cwd>.
      │  │  └─ has_root_flag()
      │  └─ run_script_in_place()  cli.py   Same mechanism as A.
      │
      ├─ C. window
      │  ├─ parse_window_arguments() cli.py Reads --port (4780), --page (web), --root (cwd).
      │  └─ open_window(repo, port, page, root)   window.py
      │     ├─ page_url()                  http://127.0.0.1:<port>/<page>
      │     ├─ server_answers(port)        GET /state, 1 s timeout. True: skip the next step.
      │     ├─ start_server_in_background() Popen "watcher.py serve --port --root" from the repo,
      │     │                               log in the temp folder, wait until server_answers.
      │     ├─ app_mode_browser()          First installed of Chrome, Chromium, Brave, Edge.
      │     └─ open_app_window(url)        Popen browser --app=<url> --window-size=380,720.
      │                                    No browser found: default browser plus a printed note.
      │
      └─ D. anything else                  print usage() to stderr, exit 1.
```

Function by function

Function	What it does

`main(argv)` cli.py	Entry point of the console script. Prints the usage for no arguments or `--help`; otherwise hands the first word to `dispatch` and turns a missing checkout into exit code 1.
`dispatch(command, arguments)` cli.py	Resolves the code folder once, then routes: discovery script, watcher script, window, or usage with exit 1.
`find_code_folder()` cli.py	Tries `find_repo`; if that raises and the bundled copies exist, returns the vendored folder instead.
`find_repo()` repo.py	Walks the candidate folders and returns the first that passes `is_repo`, else raises `RepoNotFound` with the message that names `PATH_FINDER_REPO`.
`candidate_folders()` repo.py	A generator giving the search order: the `PATH_FINDER_REPO` override if set, the parent of the package, the current directory, then every parent of it.
`is_repo(folder)` repo.py	True only when both `discovery/discover.py` and `viz/watcher.py` are files inside the folder.
`vendored_copies_present()` repo.py	`is_repo` applied to `path_finder/_vendor`, the copies the wheel carries.
`discover_script(folder)`, `watcher_script(folder)` repo.py	Build the path of each script inside the chosen folder.
`has_root_flag(arguments)` cli.py	True when the typed arguments already contain `--root` or `--root=…`.
`with_default_root(command, arguments)` cli.py	For `list`, `done`, `serve`, and `map` without a root flag, appends `--root` and the current directory; otherwise returns the arguments unchanged.
`run_script_in_place(script, command, arguments)` cli.py	Sets `sys.argv` as if the script had been typed with those arguments and runs the file as `__main__`, so the script's own argument parser and commands take over.
`usage()` cli.py	Builds the help text: the command lists from the same tuples `dispatch` uses, the root rule, and two live lines from `data_dir_description` saying which code and data directory were found.
`data_dir(code_folder)`, `data_dir_description()` repo.py	The package's copy of the data-directory rule (`PATH_FINDER_HOME`, else the checkout's `discovery/`, else `~/.path_finder`) and the two help lines built from it.
`parse_window_arguments(arguments)` cli.py	An argparse parser for `window` only: `--port` (4780), `--page` (web), `--root` (current directory).
`open_window(repo, port, page, root)` window.py	Checks for a watcher on the port, starts one from the repo if needed, then opens the page in an app-mode browser window.
`page_url`, `server_answers`, `server_log_path`, `start_server_in_background`, `app_mode_browser`, `open_app_window` window.py	The six helpers behind `open_window`: the URL, the 1 s probe of `/state`, the log file in the temp folder, the background `serve` process, the browser lookup, and the launch with `--app` and `--window-size`.

The three knobs, kept apart

- **Which code runs**: found by `find_code_folder`. `PATH_FINDER_REPO` overrides it. Normally unset.
- **Which project is watched**: the folder you run from, or `--root`. A session is shown only when its start folder sits under that root.
- **Where data is written**: `~/.path_finder`, or the checkout's `discovery/` when running inside one, or `PATH_FINDER_HOME` to override.

Worked examples

You type	What happens
`path-finder list` in back_to_the_future	Code found (checkout or vendored). `list` is a root command with no flag, so the script receives `list --root /…/back_to_the_future` and prints only that project's sessions.
`path-finder list --root /tmp/x`	Flag already present, arguments pass through as typed.
`path-finder identify --role builder`	Not a root command, nothing added. The discovery script writes the identity file to the data directory.
`path-finder window`	Probe 4780. If silent, start `serve --port 4780 --root <cwd>` in the background and wait for it. Open Chrome with `--app=http://127.0.0.1:4780/web` at 380 by 720.
`PATH_FINDER_REPO=/other/clone path-finder serve`	That clone is the first candidate; if it holds both scripts it wins, otherwise the search continues as if unset.
`path-finder --help`	Usage text plus the resolved code folder and data directory, which makes it a quick diagnostic.

Refactor notes for this layer

- Move `find_code_folder` from `cli.py` into `repo.py` beside `find_repo`, so the "checkout, else vendored" rule lives in one file.
- The data-directory rule exists three times: `repo.data_dir`, `discover.resolve_data_dir`, `watcher.resolve_data_dir`. One shared module, or one copy that the other two import, removes the risk of them drifting apart.
- `window` defines its own `--root` default; routing it through `with_default_root` would state the rule once.
- `dispatch` resolves the code folder before checking whether the command is valid, so an unknown command pays for the lookup. Checking the command first is a free improvement.